Joseph Marsh

Custom Quadrature Encoder Design and Integration

September 2024

1. Introduction

My goal for this project was to create a quadrature encoder from scratch that accurately measures the rotational position of a motor shaft and integrates it into a control system to move the motor in line with the encoder. I am creating this project in order to help others better understand encoders on a physical level. While it would be easy to use an encoder to accomplish this, I think it is incredibly useful to show how the encoder disc and sensors work in a large scale physical manner. My personal goal, which extends beyond this outlined goal, is to create this using only tools that any person would have at home. This means no 3d printing, no woodworking, and no metalworking needed, giving accessibility to anyone who wants to understand encoders. I will publish this on YouTube for anyone to see to ensure everyone has access to this process and information. Understanding encoders is extremely useful for anyone wanting to understand how our world works. Encoders are in printers, robots, airplanes, medical devices, and even everyday things like the knob on your stereo.

In order to do this, I needed to create the quadrature encoder using fundamental sensing techniques. My encoder disc rotates with alternating segments of see through and opaque plating, which must have two concentric sets of these plating segments. In my encoder disc, I have the “first” set on the outside of the encoder disc and the “second” set on the inside of the encoder disc exactly 1/32nd of a circle(11.25 degrees) after the “first” set. This arrangement will enable the arduino to generate two distinct signals that are 90 degrees out of phase. This allows the system to determine whether the disc is moving forwards or backwards.

My sensor will be accomplished using two laser pointers and two photoresistors. Each laser is pointed directly at a corresponding photoresistor and will be blocked by the rotating encoder disc. This allows the photoresistors to go from a very low light value to a very high light value in an extremely short amount of time. This means that the lighting will need to be held in place to ensure consistent detection. Finally, I must put together my circuit by using a pull-up resistor in the same circuit as the photoresistors in order to accomplish this on the hardware side. Using these pull-up resistors will allow the signals to stabilize by making the low to high signals much more clear on the processing end.

Finally, all of this data will be processed in code through my arduino. I will program the arduino to interpret the signals we get from the quadrature encoder to spin a stepper motor. The system will constantly be reading the encoder’s output and using this output to determine the movement the motor needs to go to.

2. Technical Requirements

a. Customer Requirements

- This system must accurately measure the rotational position of a motor shaft and provide this data to control the movement of a motor.
- The encoder must have a minimum of 8 positions and must be able to dictate the direction of motion, and drive the motor in the direction of the encoder disc.
- The customer does not want any built-in encoder, which means the encoder must be built by hand with materials available to the designer.
- This system must be demonstrated in video
- A report must be written about the system describing the process, technical requirements, hardware, software, and validation.

b. Technical Requirements

- Encoder Disc:
  - The disc must have 8 distinct positional states.
  - The user must have a piece of cardboard that is at least 6 inches by 6 inches in order to have a base for cutting out the encoder disc by hand.
- Laser:
  - The center of the laser beam must directly hit the center of the photoresistor.
  - The lasers must be fastened in place using duct tape in order to accurately detect the positional state of the encoder.
- Tools and Manufacturing
  - The user must have a tool to cut the cardboard. The demonstration project uses an exacto-knife.
  - The user must have a working Arduino Uno in order for the hardware to function properly.
  - The user must have one stepper motor and one stepper motor driver as the motor being rotated in this demonstration.
  - The user must have at least six male to female wires, and at least eight male to male wires to ensure all connection requirements are met.
  - The user must have a breadboard to connect the physical components.
  - The user must have two 10k ohm pull-up resistors in order to allow for a more stable signal output.

    - The user must have a long, thin cylindrical item that can stick into the center of the encoder disc. This item is used to ensure the encoder disc stays in place during rotation.
- Motor control:
    - The motor must be able to move the exact number of rotations completed by the encoder disc in either direction.

c. Engineering Specifications

- Encoder Disc:
    - The encoder disc must be exactly 6 inches in diameter.
    - The encoder disc must have 8 evenly spaced distinct rectangular holes on the outside that sit 1/4th of an inch from the edge of the disc
    - The outer rectangular holes must be 3/4sth of an inch wide(tangential to the circle) and 1/2 inch long(radial to the circle)
    - There must be a ¼ inch radial separation between the lower edge of each outer hole and the upper edge of each inner hole.
    - The inner rectangular holes must be ½ inch long by 3/4ths inch wide.
    - The disc must have two rings around the center. The radial centerline of the inner ring must be 11.25 degrees offset relative to the outer ring in order to create a 90 degree phase shift
- Lasers:
    - Laser must be 1mW
- Tools and Manufacturing:
    - The user must have a cardboard box that is at least 4 inches tall, 6 inches long, and 8 inches wide in order to house the lasers, encoder disc, and arduino board.
    - The thin cylindrical connection tool(I used a thin paintbrush) must be at least 6 inches long and between 0.1 and 1 inch in diameter and it must stick directly into the center of the cardboard encoder disc.
- Hardware and Power
    - The stepper motor must be a 28BYJ-48
    - The stepper motor driver must be a ULN2003 Driver Board

3. System Definition - Hardware

Note: The objects I've used(cardboard, photoresistors, lasers, etc) are not exact methods, and I'm well aware of this. However, they are great for the purpose of this project. My intent of this project is to get new learners a chance to learn more about Arduino and encoders on a physical level, without needing the 10 week background of this course.

Hardware Components Used:

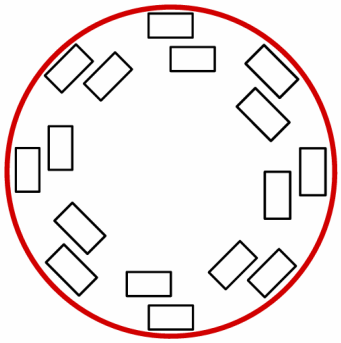
1. **Cardboard Box:**
    - **Purpose:** Serves as the base structure to house and support all other components. Modified into a table-like setup to mount the encoder disc and laser sensors.
    - **Modification:** Five sides of the box are cut away, leaving the corners intact to create a sturdy base with legs. The top of the box is cut to allow the paintbrush to spin the encoder disc.
2. **Encoder Disc:**
    - **Material:** Cardboard, cut from one of the box sides.
    - **Specifications:**
        - Diameter: 6 inches
        - Features: 8 evenly spaced rectangular holes, with specific dimensions listed in the engineering specifications piece. Listed again here for clarity:
            1. The encoder disc must have 8 evenly spaced distinct rectangular holes on the outside that sit 1/4th of an inch from the edge of the disc
            2. The outer rectangular holes must be 1/4th of an inch wide(tangential to the circle) and 1/2 inch long(radial to the circle)
            3. There must be a ¼ inch radial separation between the lower edge of each outer hole and the upper edge of each inner hole.
            4. The inner rectangular holes must be ½ inch long by 1/4th inch wide.
            5. The disc must have two rings around the center. The radial centerline of the inner ring must be 11.25 degrees offset relative to the outer ring in order to create a 90 degree phase shift
        - The disc is manually spun using a long, thin paintbrush(or other long thin object, could even be a pencil) inserted through a hole in the top of the cardboard box.
3. **Paintbrush:**
    - **Purpose:** Used to manually spin the encoder disc. Doesn't have to be a paintbrush. I used a paintbrush in my demonstration, but many long, thin objects would work, even a pencil. It is vertically through a hole in the box's top to ensure stability and accurate rotation.
4. **Stepper Motor:**

- **Model:** 28BYJ-48
- **Purpose:** Drives the physical movement based on encoder readings.

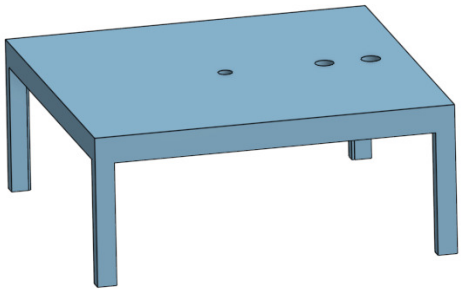
5. **Stepper Motor Driver:**
    - **Model:** ULN2003 Driver Board
    - **Purpose:** Controls the stepper motor based on signals from the Arduino.
6. **Arduino Uno:**
    - **Purpose:** The brain of the operation. The Arduino does all the computation required for this to work.
    - **Pin Utilization:**
        - Pin 8 to pin 1N1 on the ULN2003
        - Pin 9 to pin 1N2 on the ULN2003
        - Pin 10 to pin 1N3 on the ULN2003
        - Pin 11 to pin 1N4 on the ULN2003
        - Analog Pin A0 to photoresistor 1
        - Analog Pin A1 to photoresistor 2
7. **Lasers:**
    - **Purpose:** Emit light to be detected by photoresistors.
    - **Setup:** Two holes are cut into the top of the box, positioned to shine lasers perpendicular to the surface of the table. Lasers are placed face down on the top of the box. One laser is positioned 3/4 inch from the edge of the encoder disc, and the other is 1 1/4 inch from the edge. This setup ensures that the lasers shine directly at the center of the photoresistors when the encoder disc is in the correct position.
8. **Photoresistors:**
    - **Purpose:** Detect changes in light caused by the encoder disc's holes.
    - **Setup:** Each photoresistor is placed directly under the corresponding laser. When the encoder disc moves and blocks or unblocks the laser light, the photoresistors will detect these changes.
9. **Resistors:**
    - **Type:** 10k ohms
    - **Purpose:** Pull-up resistors used in the photoresistor circuits to stabilize signal readings.
10. **Breadboard and Wires:**
    - **Purpose:** Used as my circuit creation method.
    - **Wiring:**
        - Male to female and male to male wires connect the Arduino to the ULN2003 Driver Board and the photoresistor setups.

Parts Diagrams:

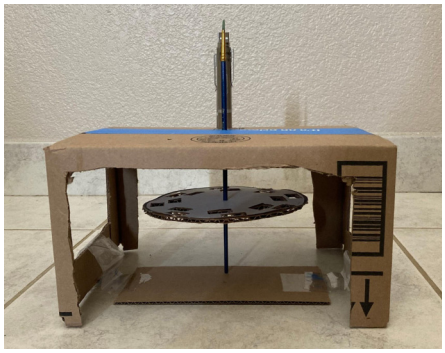
Encoder Disc Example



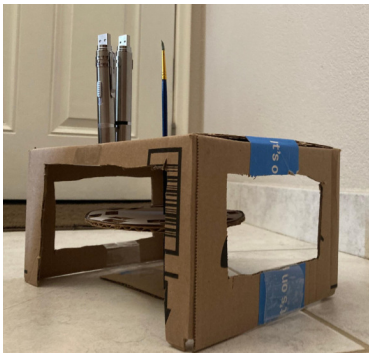
Cardboard Table cutout 3D model



Housing: Front View

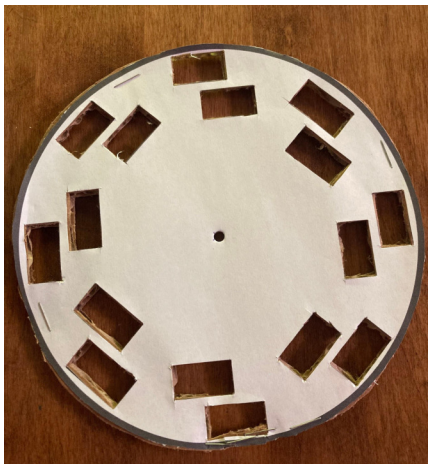


Housing: Side View

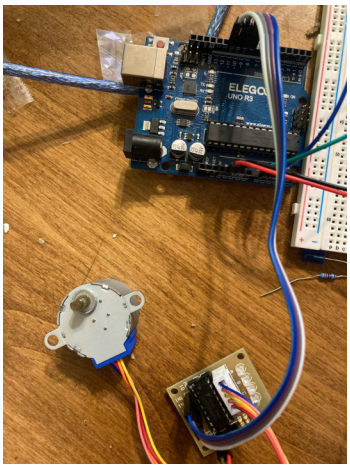


Housing: Top View

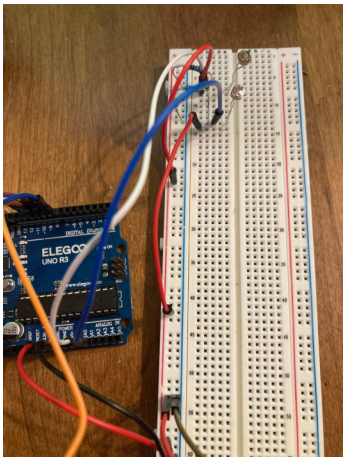
Cardboard Encoder Disc

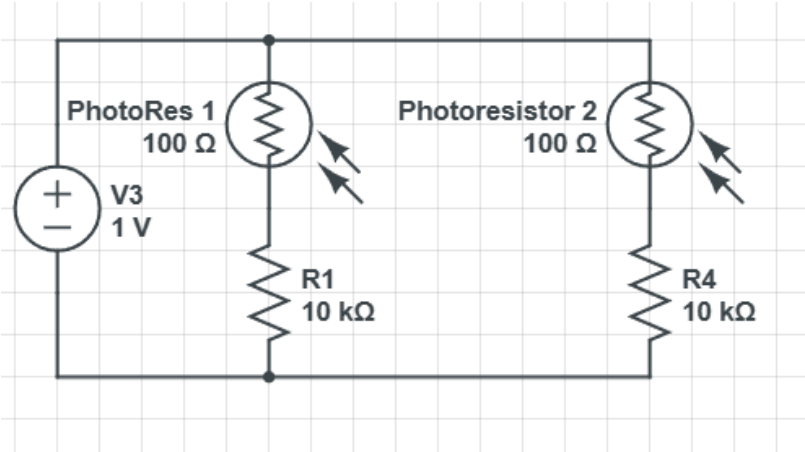


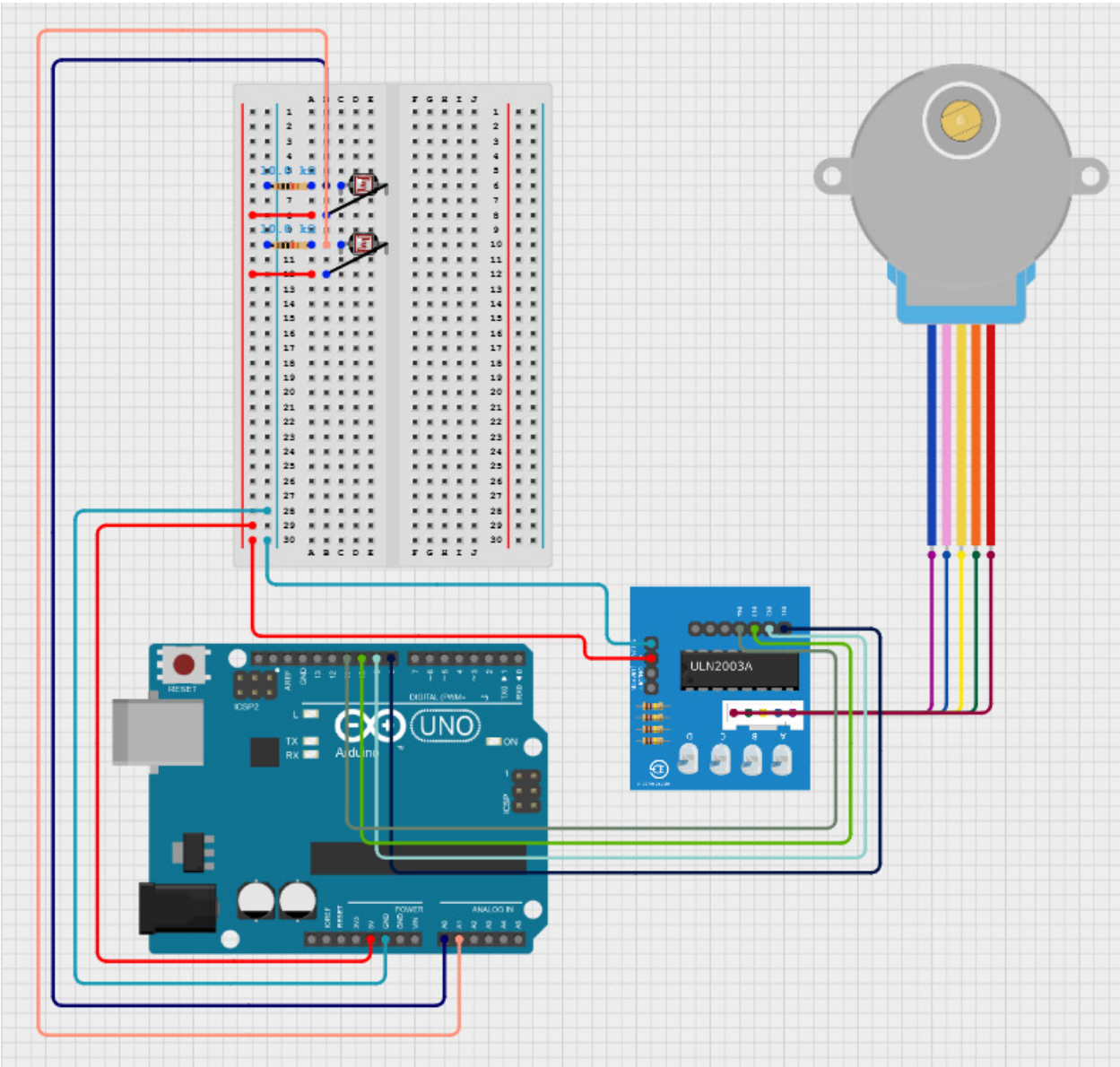
Stepper Motor Circuit



Photoresistor Circuit




PhotoRes 1
100 Ω
Photoresistor 2
100 Ω
V3
1 V
R1
10 kΩ
R4
10 kΩ


ULN2003A
UNO
Arduino
RESET
ICSP2
POWER
ANALOG IN

Bill of Materials:

Component	Description	Quantity
Cardboard Box	Physical base for the stabilization of lasers and encoder disc	1
Cardboard encoder disc	6 inch diameter, used as physical encoder	1
Paintbrush	Used to rotate encoder disc	1
28BYJ-48 Stepper Motor	Used to demonstrate functionality of encoder	1
ULN2003 Driver Board	To control Stepper motor	1
Arduino Uno	microcontroller	1
Lasers	For changing voltage state within photoresistors, 1mW	2
Photoresistors	Detect light coming from lasers. Use as high/low voltage state signals	2
Resistors	10k ohms, used as pull-up resistors	2
breadboard	Base for the circuit creation	1
Male to female wires	Connections between driver board and arduino	6
Male to male Wires	Connections between breadboard and arduino	8

4. System Definition - Software

Code:

```
#include <Stepper.h>

const int P1Pin = A1;
const int P2Pin = A0;

const int threshold = 800;
const int StepsPerRevolution = 2048;
const int StepsPer8thRevolution = 256;

int P1Value; //set up photoresistor values
int P2Value;
int lastP1Val = 0; //initialize at 0 since I don't have a starting value for
this
int lastP2Val = 0;

Stepper stepper(StepsPerRevolution, 8, 10, 9, 11); //8,9,10,11 didn't work,
this allowed accurate rotation detection

void setup() {
  stepper.setSpeed(5);

  pinMode(P1Pin, INPUT);
  pinMode(P2Pin, INPUT);

  Serial.begin(9600);
}

void loop() {
  P1Value = analogRead(P1Pin);
  P2Value = analogRead(P2Pin);

  // Check if photoresistor1 has crossed the threshold
  if (lastP1Val < threshold && P1Value >= threshold) {
    // Check the state of photoresistor2 at the transition
    Serial.print("Photoresistor 1:"); //Printing for troubleshooting
    Serial.println(P1Value);
    Serial.print("Photoresistor 2:");
    Serial.println(P2Value);
    if (P2Value < threshold) {
      stepper.step(-StepsPer8thRevolution );
      Serial.println("Counterclockwise");
    }
    else {
      stepper.step(StepsPer8thRevolution );
      Serial.println("Clockwise");
    }
  }

  // Update last values
  lastP1Val = P1Value;
  lastP2Val = P2Value;
}
```

Overview:

The following sections will go in detail on each of my Algorithmic and Software Choices, as well as an outline for why I wrote my code how I did and how future students can use this architecture to write out their own code. I will explain why I used either polling over interrupts as well as a step by step process through the algorithms within writing each section of the code.

Polling Usage:

For this project, I've decided to use polling. While I know its limitations in quicker systems, I believe polling is a great and simple introduction to new learners on Arduinos. Polling is a fantastic use in this project as the encoder disc is being spun relatively slow, which means the sensor can catch all of the distinct positions in the encoder disc. If I had an encoder disc with 32 or more positions and I was spinning it at 200+ rpm, polling would be completely unfeasible. For the purpose of this project – a low resolution, slow spinning handmade encoder disc – polling is perfectly sufficient. Additionally, with the relatively simple code structure I created, the learner can check the photoresistor values and determine direction without worrying about the complexities of interrupt handling.

Serial Initialization and Integer Setups:

I used the serial to initialize communication for troubleshooting and for some more clarity for learners to understand the step logic I'm using. I just printed out the direction of rotation each time there was a rotation ticked. Additionally on each rotation tick, I printed out each resistor value at the time of the rotation.

Stepper.h Library:

This library is made for stepper motors and simplifies a massive amount of the code for stepper motors. I simply have to initialize my stepper motor with the steps per revolution and pin configuration and it sets up nearly everything I need for my code. I set the speed of the stepper to "5" after trial and error. I determined that 10 was too fast and caused the motor to stop working entirely and 3 was annoyingly slow. Finally, I just used the step function to rotate 1/8th of a direction each time the tick triggered.

Photoresistor Setup:

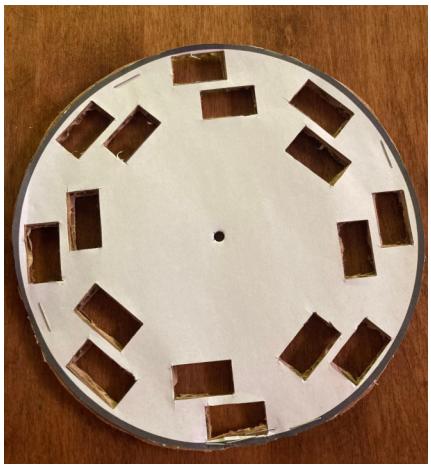
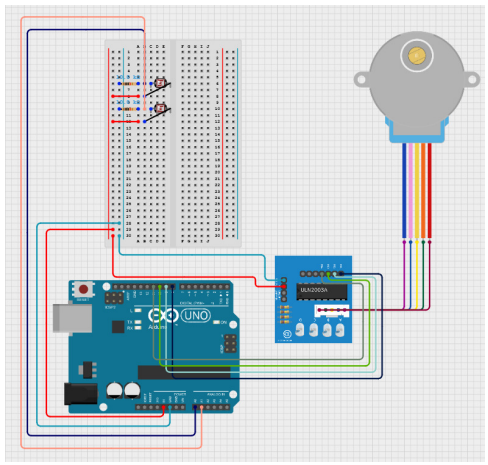
I've connected 2 photoresistors to Pins A0 and A1, which are read by the Arduino to detect changes in light. I've set a threshold of 800, which is about 100 below the max 900 value that they reach under full laser light.

Step Logic:

My logic solely is based on the moment photoresistor 1 transitions from a low state to a high state above my threshold. The way I do this is by saving the last state value and checking whether the last state value was low or high. If the last state value was low and the current state is high, that means that the encoder disc must have transitioned from blocking to not blocking the light. Low->low and High-> high do nothing because they are not state changes. High-> low also does nothing. When the state of photoresistor 1 goes from low to high, it does a check. The state change of 1 indicates rotation is happening, but it gives not information on the direction. Photoresistor 2 gives direction information. If the value of photoresistor 2 is below the threshold, it determines that the encoder disc is moving counterclockwise. Otherwise, it must be moving clockwise. This is because when moving counterclockwise(on my disc) the outer ring goes from low to high before the inner ring does so the inner photoresistor reads a low signal. Similarly, when moving clockwise(on my disc) the outer ring goes from low to high after the inner ring does so the inner photoresistor reads a high signal. After this happens, I update the previous state to the current state and the process continues. I also initialized previous state at 0 since it otherwise wouldn’t have an initial value.

5. Results - Validation

8 Position Encoder Disc Validation and no Built-in Encoder Validation

This image of my encoder disc is visual validation that my encoder disc has 8 distinct positions. Additionally, I have shown multiple images of my setup. None of these include a built-in encoder module. Above I also show my fully functioning self-made encoder wiring.

Gear Ratio/Rotation Validation (Validation of Rotation Accuracy)

I've done visual inspections of my encoder disc. I've ensured that it has 8 holes on the outside and 8 holes on the inside. This means that each step must be 1/8th of a rotation. To calculate this on the motor, we must figure out the gear ratio and the steps per revolution. We know the gear ratio is 64:1 and the total number of full steps for one revolution is 32. So, 32*64=2048 ticks per revolution. Dividing this by 8 gives a step count of 256, which is the step count in my code. Additionally, I have visually validated that any amount of turns on the encoder disc accurately represents the real number of turns.

```
const int threshold = 800;
const int StepsPerRevolution = 2048;
const int StepsPer8thRevolution = 256;
```

$$32 \times 64 = 2048\ ticks\ per\ revolution$$

Threshold Validation(for validation of brightness threshold)

I set my threshold at 800, which is in between the full brightness of the laser on the photoresistor and the ambient light of the background. Below is a demonstration of the photoresistor values with and without the laser light. Photoresistor 2 is exposed to ambient light while photoresistor 1 is exposed directly to the laser. I've used a threshold of 800 because when

I set it to 900 or above, my lasers would occasionally not hit that value. Additionally, when I set the threshold below 500, occasionally light would seep from one laser to the opposite photoresistor, which caused false positives. Setting my threshold to 800 was a great medium that eliminated all background light and also allows the laser to always be detected.

```
Photoresistor 1:1012
Photoresistor 2:149
```

Demonstration of Clockwise Vs Counterclockwise Movement

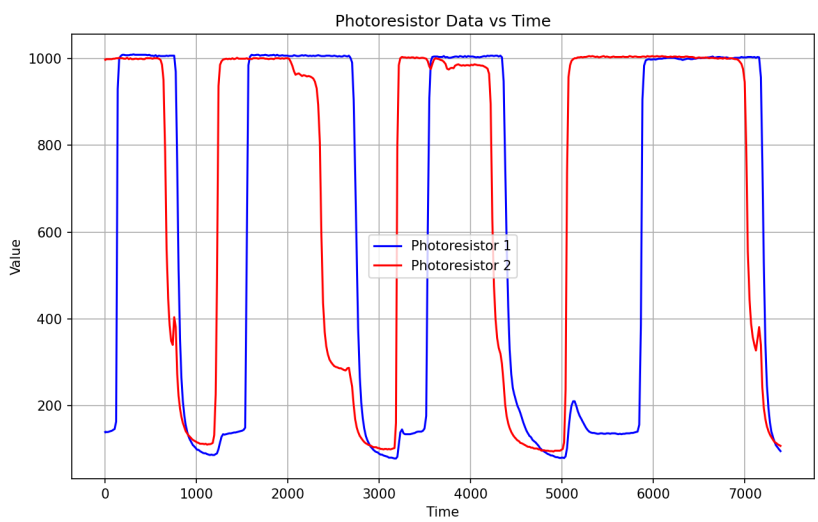
Image: Demonstration of Clockwise movement. Last photoresistor 1 value is below threshold, and current photoresistor 1 value is above threshold. This indicates a change from low to high. Additionally, photoresistor 2 at this time is high, which leads to a 1,1 state at the time of photoresistor 1 crossing the threshold, which indicates clockwise motion as stated in my algorithms section.

```
Last Photoresistor 1:794
Photoresistor 1:822
Photoresistor 2:999
Clockwise
Last Photoresistor 1:784
Photoresistor 1:834
Photoresistor 2:998
Clockwise
Last Photoresistor 1:777
Photoresistor 1:804
Photoresistor 2:997
Clockwise
Last Photoresistor 1:793
Photoresistor 1:817
Photoresistor 2:997
Clockwise
Last Photoresistor 1:777
Photoresistor 1:803
Photoresistor 2:996
Clockwise
Last Photoresistor 1:794
Photoresistor 1:840
Photoresistor 2:999
```

Image: Demonstration of Clockwise movement. Last photoresistor 1 value is below threshold, and current photoresistor 1 value is above threshold. This indicates a change from low to high. Additionally, photoresistor 2 at this time is low, which leads to a 1,0 state at the time of photoresistor 1 crossing the threshold, which indicates counterclockwise motion as stated in my algorithms section.

```
Last Photoresistor 1:798
Photoresistor 1:835
Photoresistor 2:296
CounterClockwise
Last Photoresistor 1:796
Photoresistor 1:826
Photoresistor 2:223
CounterClockwise
Last Photoresistor 1:770
Photoresistor 1:808
Photoresistor 2:433
CounterClockwise
Last Photoresistor 1:780
Photoresistor 1:826
Photoresistor 2:298
CounterClockwise
Last Photoresistor 1:784
Photoresistor 1:804
Photoresistor 2:230
CounterClockwise
Last Photoresistor 1:783
Photoresistor 1:802
Photoresistor 2:186
CounterClockwise
```

Phase Shift Demonstration(For CW/CCW rotation measurements)



I mapped my photoresistor data with time to demonstrate my phase shift. As you can see, since I have a cardboard cutout, not all phase shifts are exactly 90 degrees, which I will discuss in my conclusion as a way to improve my project in the future. Additionally, I was spinning this by hand which could have caused inaccuracies in the time data. You will notice that blue always goes low to high slightly after the red signal, which still acts as a phase shift and still works with the encoder we are using.

6. Conclusion

I would ultimately call this project an absolute success(well, more like a quadrature success ;). I successfully built the 8 position quadrature encoder disc, successfully measured rotation with the encoder, translated it to the stepper motor, and I made it simple enough for new learners to understand the project.

I encountered SO MANY issues in this project along the way. First off, I really struggled with my understanding of circuits. I began the project using a wide array of components that increasingly got more complicated. My circuit ended up getting so complicated that I accidentally misplaced a wire and blew up an LED. It was funny… but also extremely embarrassing… I also struggled incredibly hard with the code. I couldn't figure out why my stepper motor was not working properly. It took me 3 days and many, many questions to realize that my stepper motor was fundamentally coded wrong.

I messed up my encoder disc. A LOT. I have about 5 iterations of my encoder disc that I deemed were unworthy of use in the project. I wanted to make something that was easy to replicate for beginners and easy to cut out. That's why I came up with my final design. The holes are square so the user can just cut out 4 lines and easily cut through the cardboard. The holes are identical sizes so the learner doesn't have to worry about cutting out small holes. Initially, I made an encoder disc with 16 positions and the holes took forever to cut out. I decided to lower it to 8 to really demonstrate the fundamentals of an encoder without having to make the learner worry about complexity.

For my next steps, I plan on continuing to refine my design to make it as easy to replicate as possible to anyone wanting to start out with an Arduino and an encoder Ideally, I'd like to make it simple enough that even a middle schooler could copy/paste my code and wire the circuit in the way I wired it. I'm going to do this by making the design of the box more clear, and making the code easy to simply copy/paste into their own IDE. Additionally, I think I'm going to add options for more advanced users to explore if they want. To do this, I'll add a CAD model of the encoder disc so people can just print out the encoder disc with their desired specifications in a 3D printer if they own one. Additionally, I'll add code with interrupts so more advanced learners will be able to enjoy a more in-depth approach. As far as my housing and setup, I could also make a CAD model of the housing for anyone to use, which would streamline the process of cutting out the housing.

Some things that I personally have learned is the importance of mapping everything out before you begin. I didn't do very well on the project plan and it really bit me in the butt. I didn't have the physical design of my project created until after my project plan was created and I really struggled with the creation of the housing. I'm very lucky that it turned out as well as it did, but it could have gone much worse. In my future projects, I really need to focus on mapping everything out before I start on it. Even if my plans have to change, having everything mapped out will save me so much time in the future.